Week4_예습과제_명재서

4장. 분류 (4.5~4.8, 4.10~4.11) -p.246 - p.326

4.5 GBM (Gradient Boosting Machine)

- **부스팅(Boosting)의 개요**: 여러 개의 약한 학습기(Weak Learner)를 순차적으로 학습-예측하면서, 잘못 예측한 데이터에 **가중치 부여**를 통해 오류를 개선해 나가는 학습 방식.
- **GBM의 원리**: 가중치 업데이트를 경사 하강법(Gradient Descent)을 이용해 수행함.
  - 오류 값 = 실제값 - 예측값. 이 오류 함수를 최소화하는 방향으로 가중치를 업데이트함.
- **특징**:
  - **장점**: 랜덤 포레스트보다 예측 성능이 뛰어난 경우가 많음.
  - **단점**: 순차적 수행으로 인해 **수행 시간이 매우 오래 걸림.** 하이퍼 파라미터 튜닝 노력이 많이 필요함.

주요 파라미터

- `loss`: 경사 하강법에서 사용할 비용 함수
- `learning_rate`: GBM이 학습을 진행할 때마다 적용하는 학습률, 약한 학습기가 순차적으로 오류 값을 보정해 나가는 데 적용하는 계수. 0~1 사이. → n_estimator 상호보완적으로 조합해 사용함.
- `subsample` :약한 학습기가 학습에 사용하는 데이터의 샘플링 비율

4.6 XGBoost (eXtreme Gradient Boosting)

- **개요**: GBM 기반이지만, 느린 수행 시간과 과적합 문제를 해결하기 위해 규제(Regularization) 기능을 추가한 알고리즘.
- **주요 장점**:
  1. **뛰어난 예측 성능**: 일반적으로 분류와 회귀 영역에서 뛰어난 성능 발휘.
  2. **GBM 대비 빠른 수행 시간**: 병렬 CPU 환경에서 병렬 학습 가능.

3. **과적합 규제**: 자체적인 L1, L2 규제 포함.
4. **조기 중단(Early Stopping)**: 예측 오류가 더 이상 개선되지 않으면 반복을 중지하여 시간 단축.

- **구현 방식**:
  - **파이썬 래퍼**: 전용 라이브러리 사용 (`DMatrix` 객체 이용).
  - **사이킷런 래퍼**: `XGBClassifier`, `XGBRegressor`를 통해 기존 사이킷런 프레임워크와 통합 사용.

🚀 하이퍼 파라미터 정리

1. 주요 일반 파라미터 (General Parameters)

일반적으로 실행 시 스레드의 개수나 silent 모드 등의 선택을 위한 파라미터입니다.

- `booster`: gbtree(tree based model) 또는 gblinear(linear model) 중 선택합니다. (기본값은 gbtree)
- `silent`: 출력 메시지를 나타내지 않을지 설정합니다. (기본값은 0이며, 1 설정 시 메시지를 출력하지 않음)
- `nthread`: CPU의 실행 스레드 개수를 조정합니다. (기본값은 CPU의 전체 스레드 사용)

---

2. 주요 부스터 파라미터 (Booster Parameters)

트리 최적화, 규제, 샘플링 등을 제어하는 핵심 파라미터 군입니다.

- `eta` [default=0.3, alias: `learning_rate`]: GBM의 학습률과 같은 파라미터입니다. 0에서 1 사이의 값을 가지며, 보통 0.01~0.2 사이의 값을 선호합니다.
- `num_boost_round`: 학습을 수행할 결정 트리의 개수입니다. (사이킷런의 `n_estimators`와 동일)
- `min_child_weight` [default=1]: 과적합 조절용입니다. 자식 노드에 필요한 모든 관측치에 대한 가중치의 합으로, 이 값이 클수록 분할을 자제합니다.
- `gamma` [default=0, alias: `min_split_loss`]: 리프 노드를 추가적으로 분할할지 결정할 최소 손실 감소 값입니다. 값이 클수록 과적합 감소 효과가 있습니다.
- `max_depth` [default=6]: 트리의 최대 깊이입니다. 0을 지정하면 깊이 제한이 없으며, 보통 3~10 사이의 값을 사용합니다.
- `sub_sample` [default=1]: 과적합 제어를 위해 데이터를 샘플링하는 비율입니다. 일반적으로 0.5~1 사이의 값을 사용합니다.

- `colsample_bytree` [default=1]: 트리 생성에 필요한 피처(Column)를 임의로 샘플링하는 비율입니다.
- `lambda` [default=1, alias: `reg_lambda`]: L2 Regularization 적용 값입니다. 피처 개수가 많을 때 적용하며, 값이 클수록 과적합 감소 효과가 있습니다.
- `alpha` [default=0, alias: `reg_alpha`]: L1 Regularization 적용 값입니다. 피처 개수가 많을 때 적용하며, 값이 클수록 과적합 감소 효과가 있습니다.
- `scale_pos_weight` [default=1]: 특정 값에 치우친 불균형 데이터 세트의 균형을 유지하기 위한 파라미터입니다.

---

3. 학습 태스크 파라미터 (Learning Task Parameters)

학습 시의 목적 함수나 평가 지표를 설정하는 파라미터입니다.

- `objective`: 최솟값을 가져야 할 손실 함수를 정의합니다. (이진 분류 시 `binary:logistic`, 다중 분류 시 `multi:softmax` 등)
- `eval_metric`: 검증 데이터 세트에 적용할 성능 평가 방법을 지정합니다.
  - `rmse`: Root Mean Square Error
  - `mae`: Mean Absolute Error
  - `logloss`: Negative log-likelihood
  - `error`: Binary classification error rate
  - `auc`: Area under the curve

4.7 LightGBM

- **개요**: XGBoost보다 더 빠르고 메모리 사용량도 적은 알고리즘.
- **리프 중심 트리 분할(Leaf-wise)**:
  - 일반적인 트리 알고리즘은 트리의 균형을 맞추는 **균형 중심 분할(Level-wise)**을 사용함.
  - LightGBM은 **최대 손실 값(Max Delta Loss)**을 가지는 리프 노드를 지속적으로 분할하여 비대칭적인 깊은 트리를 생성함. 이 방식이 예측 오류 손실을 더 효과적으로 줄임.
- **특징**:
  - **장점**: 속도가 매우 빠르고 대용량 데이터 처리에 적합함.
  - **단점**: 데이터 개수가 적을 경우(약 10,000건 이하) 과적합이 발생하기 쉬움.

- **주요 파라미터**
  - **num_iterations (n_estimators)** : 반복 수행할 트리의 개수

    → 값이 클수록 성능이 좋아질 수 있지만, 너무 크면 과적합 발생 가능
  - **learning_rate** : 학습률 (0~1 사이 값)

    → 작게 설정 + 트리 개수 증가 = 일반적으로 성능 향상 (단, 학습 시간 증가)
  - **max_depth** : 트리의 최대 깊이

    → 0 또는 음수면 제한 없음

    → LightGBM은 leaf-wise 방식이라 깊이가 깊어지기 쉬움
  - **min_data_in_leaf (min_child_samples)** : 리프 노드가 되기 위한 최소 데이터 수

    → 값이 클수록 과적합 방지
  - **num_leaves** : 한 트리가 가질 수 있는 최대 리프 개수

    → 클수록 모델 복잡도 증가
  - **boosting** : 부스팅 방식 선택
    - `gbdt` : 기본 Gradient Boosting
    - `rf` : Random Forest 방식
  - **bagging_fraction (subsample)** : 데이터 샘플링 비율

    → 과적합 방지용 (0~1 사이 값)
  - **feature_fraction (colsample_bytree)** : 피처 샘플링 비율

    → 트리 생성 시 일부 피처만 사용 (과적합 방지)
  - **lambda_l2 (reg_lambda)** : L2 정규화 강도

    → 값이 클수록 과적합 감소
  - **lambda_l1 (reg_alpha)** : L1 정규화 강도

    → 불필요한 피처 제거 효과 있음

Learning Task 파라미터

- **objective** : 최소화할 손실 함수 정의

하이퍼 파라미터 튜닝 방안

→ learning_rate를 작게 하면서 n_estimator를 크게 하기.

→ 과적합을 제어하기 위해 reg_lambda, reg_alpha와 같은 규제를 적용.

→ 학습 데이터에 사용할 피처의 개수 / 데이터 샘플링 레코드 개수를 줄이기 위해 colsample_bytree, sumsample 파라미터 적용하기.

4.8 베이지안 최적화 (Bayesian Optimization)

- **개요**: 목적 함수 식을 제대로 알 수 없는 블랙 박스 형태의 함수를 최대 또는 최소로 만드는 최적의 하이퍼 파라미터를 찾는 기법. / 새로운 데이터를 입력 받았을 때 최적 함수를 예측하는 사후 모델을 개선해 나가면서 최적 함수 모델을 만들어 냄.
- 베이지안 확률에 기반을 두고 있는 최적화 기법. → 베이지안 확률? 새로운 사건의 관측이나 새로운 샘플 데이터를 기반으로 사후 확률을 개선해 나감.
- **사용 이유**: GridSearchCV처럼 모든 조합을 다 시도하기에는 시간이 너무 많이 걸리는 대용량 모델 최적화에 사용함.

- **베이지안 최적화의 요소**
  1. **대체 모델(Surrogate Model)**: 획득 함수로부터 최적 함수를 예측.
  2. **획득 함수(Acquisition Function)**: 이전 학습 결과를 바탕으로 다음으로 시도할 파라미터를 결정.

→ 대체 모델은 획득 함수가 계산한 하이퍼 파라미터를 입력받으면서 점차적으로 개선되며, 개선된 대체 모델을 기반으로 획득 함수는 더 정확한 하이퍼 파라미터를 계산할 수 있게 됨.

[베이지안 최적화 (Bayesian Optimization) 단계]

1. **초기 샘플링 (Initial Sampling)**
   - 하이퍼파라미터를 몇 개 랜덤으로 선택하여 모델을 실행
   - 각 파라미터 조합에 대한 성능을 측정
     → 전체 탐색 공간에 대한 초기 정보 확보
2. **확률 모델 생성 (Surrogate Model)**
   - 지금까지의 실험 결과를 바탕으로 성능을 예측하는 모델 생성
   - 어떤 파라미터에서 성능이 좋을지 확률적으로 추정
     → 일반적으로 가우시안 프로세스(Gaussian Process) 사용

3. **다음 탐색 지점 선택 (Acquisition Function)**
    - 성능이 좋을 것으로 예상되는 지점 + 아직 탐색하지 않은 지점을 고려
    - 탐색(Exploration)과 활용(Exploitation)을 균형 있게 반영
      → 가장 유망한 하이퍼파라미터 조합 선택
4. **모델 평가 및 업데이트 (Iteration)**
    - 선택된 파라미터로 모델 학습 후 성능 평가
    - 결과를 다시 확률 모델에 반영하여 업데이트
    - 위 과정을 반복 수행

- **HyperOpt**: 파이썬의 대표적인 베이지안 최적화 라이브러리. `fmin()`, `hp` (Search Space) 등을 사용하여 최적값 도출.

4.10 스태킹 앙상블 (Stacking Ensemble)

- **개요**: 개별 모델의 예측 결과값을 다시 학습 데이터로 사용하여 최종 모델(메타 모델)이 학습하고 예측하는 기법.
- **구조**:
    1. **Step 1**: 개별 알고리즘(Base Models)들이 각각 데이터를 학습하고 예측 결과를 도출함.
    2. **Step 2**: 위에서 나온 결과들을 합쳐서 새로운 데이터 세트를 만들고, 이를 **최종 메타 모델**이 학습하여 최종 예측을 수행함.
- **CV 세트 기반 스태킹**: 데이터 세트가 적을 때 발생하는 과적합을 방지하기 위해, 교차 검증(Cross Validation)을 통해 예측 데이터를 생성하는 방식.

4.11 불균형한 데이터 세트 처리 (신용카드 사기 검출)

- **문제점**: 레이블 분포가 극도로 불균형(예: 사기 0.1%, 정상 99.9%)할 때, 일반적인 정확도 지표는 모델의 성능을 제대로 대변하지 못함.
- **해결 방법**:
    1. **언더 샘플링**: 많은 쪽의 데이터를 줄여 균형을 맞춤 (데이터 유실 우려).
    2. **오버 샘플링**: 적은 쪽의 데이터를 증식시킴.
- **SMOTE (Synthetic Minority Over-sampling Technique)**:

    - 적은 데이터 세트의 개별 데이터들의 **K-최근접 이웃(K-Nearest Neighbor)** 을 찾아 그 사이의 값을 생성하여 데이터를 증식시키는 방식.
    - 단순히 데이터를 복사하는 것보다 과적합 방지에 효과적임.
- **평가 지표**: 불균형 데이터에서는 Accuracy보다는 **Precision(정밀도), Recall(재현율), ROC-AUC**가 훨씬 중요한 평가지표가 됨.